the DOM ID that will be the target of the resulting Turbo Stream, meaning it is the DOM ID to which the HTML will be appended (the default value is the pluralized name of the model class). Any further arguments—we don't have any here—are evaluated as though they were being passed to `render`, `partial:`, `locals:`, and whatever else. If no render path is provided, we use the default Rails partial class for the path, which in our case is `app/views/favorites/_favorite`. The `locals` array always has `self` merged into it as the class name, so even though we don't specify a `locals` array, the partial is called with `{favorite: self}`.

That's a lot for one line of code.

We can now also parse the method that goes with the final callback —`after_destroy_commit`—and is executed when an ActiveRecord object is deleted from the database: `broadcast_remove_to`. So: `broadcast`, then the `remove` action, `to` the named stream, which is still `user, :favorites`. The `broadcast_remove_to` method assumes that the target DOM ID is `dom_id(self)`, or for us, `favorite_<ID>`. This is good because that's what we used. We don't need to do the removal in the background because there's no render step, so the ActionCable broadcast won't noticeably delay the original action.

With this code in place, our cable *almost* works. Open two browsers and log in as the same user—with Redis running if you have the app configured to use Redis. Make one of the concerts a favorite. You'll notice that the other browser gets the concert sliding in to the favorites with the same CSS animation, and the original browser gets the concert sliding in—twice. It gets the append from the Turbo Stream from the original form response and then it also gets the append from the ActionCable broadcast.

Fixing that is potentially easy enough. We can change the `app/views/favorites/create.turbo_stream.erb` file to remove the `turbo_stream.append` call, since we know that the append will happen via the ActionCable route. While it may be easy to make this change, it can be tough to keep track of all the changes when some changes come from a form submission and others come via ActionCable. It might be easier to have everything happen via ActionCable.